

\$limit (aggregation stage)

Definition

`$limit` Limits the number of documents passed to the next stage in the pipeline.

Compatibility

`$limit` You can use `$limit` for deployments hosted in the following environments:

- MongoDB Atlas: The fully managed service for MongoDB deployments in the cloud
- MongoDB Enterprise: The subscription-based, self-managed version of MongoDB
- MongoDB Community: The source-available, free-to-use, and self-managed version of MongoDB

Syntax

The `$limit` stage has the following prototype form:

```
{ $limit: <positive 64-bit integer> }
```

`$limit` takes a positive integer that specifies the maximum number of documents to pass along.

Starting in MongoDB 5.0, the `$limit` pipeline aggregation has a 64-bit integer limit. Values passed to the pipeline which exceed this limit will return a `invalid argument` error.

Behavior

Using \$limit with Sorted Results

If using the `$limit` stage with any of:

- the `$sort` aggregation stage,

- the `sort()` method, or
- the `sort` field to the `findAndModify` command or the `findAndModify()` shell method,

be sure to include at least one field in your sort that contains unique values, before passing results to the `$limit` stage.

Sorting on fields that contain duplicate values may return an inconsistent sort order for those duplicate fields over multiple executions, especially when the collection is actively receiving writes.

The easiest way to guarantee sort consistency is to include the `_id` field in your sort query.

See the following for more information on each:

- Consistent sorting with `$sort` (aggregation)
- Consistent sorting with the `sort()` shell method
- Consistent sorting with the `findAndModify` command
- Consistent sorting with the `findAndModify()` shell method

Examples

Consider the following example:

```
db.article.aggregate([
  { $limit : 5 }
]);
```

This operation returns only the first 5 documents passed to it by the pipeline. `$limit` has no effect on the content of the documents it passes.

`$limit`

`Limit()`

limits the number of returned documents to `10` :

To use the MongoDB .NET/C# driver to add a `$limit` stage to an aggregation pipeline, call the `Limit()` method on a `PipelineDefinition` object.

The following example creates a pipeline stage that limits the number of returned documents to 10 :

```
var pipeline = new EmptyPipelineDefinition<BsonDocument>()  
    .Limit(10);
```

`$limit`

limits the number of returned documents to 10

To use the MongoDB Node.js driver to add a `$limit` stage to an aggregation pipeline, use the `$limit` operator in a pipeline object.

The following example creates a pipeline stage that limits the number of returned documents to 10 . The example then runs the aggregation pipeline:

```
const pipeline = [{ $limit: 10 }];  
  
const cursor = collection.aggregate(pipeline);  
return cursor;
```

When a `$sort` precedes a `$limit` and there are no intervening stages that modify the number of documents, the optimizer can coalesce the `$limit` into the `$sort` . This allows the `$sort` operation to only maintain the top `n` results as it progresses, where `n` is the specified limit, and ensures that MongoDB only needs to store `n` items in memory. This optimization still applies when `allowDiskUse` is `true` and the `n` items exceed the aggregation memory limit.

Learn More

To learn how to use `$limit` in a full example, see the [Filter Data](#) tutorial.